

CPF | Core Platform Framework

Business Platform Documentation

CPF 아키텍처설계서

Module Owner, Topology, Domain Invocation, Gateway, Batch, DB 와 운영 경계를 한 구조로 설명합니다.

문서 기준	Documentation Harness v2.1.0
기준 Source	master 054d894b47f4be8323439dc6f9e58b7d8b60fe54
기준일	2026-08-26

이 문서는 최신 Source 의 공개 계약과 운영 경계를 기준으로 작성하며, 실행하지 않은 검증을 성공으로 기록하지 않습니다.

목차

역할과 질문을 기준으로 필요한 절을 빠르게 찾습니다.

1. Architecture 원칙과 Module Ownership

- 1.1 Owner 와 책임
- 1.2 Dependency 방향
- 1.3 Public/SPI/Internal 경계

2. CPF 전체 Architecture Map

- 2.1 Business Domain
- 2.2 Framework/Starter/Common
- 2.3 Gateway/Backoffice
- 2.4 Batch
- 2.5 DB/Operations

3. Same JVM · Remote · Domain Invocation

- 3.1 Same JVM
- 3.2 Remote/MSA
- 3.3 Domain Invocation
- 3.4 Self-HTTP 금지

4. Context · Security Boundary

- 4.1 System6
- 4.2 Trust/Security Boundary
- 4.3 Operation/Instance Identity

5. Gateway · Backoffice

- 5.1 Gateway Optionality
- 5.2 Backoffice/Owner Domain 경계

6. Batch Architecture

- 6.1 Control Plane/Scheduler/Worker
- 6.2 Center-Cut/Agent
- 6.3 Lease/Fencing/Heartbeat

7. Persistence · DB Ownership

- 7.1 DB Owner
- 7.2 Oracle/PostgreSQL/MariaDB
- 7.3 Migration/Upgrade/Rollback 경계

8. Observability · Multi-instance · HA

- 8.1 Log/Trace/Timeline
- 8.2 Multi-instance
- 8.3 HA/Failure Domain

9. Deployment · 금지 패턴 · Decision

- 9.1 Deployment Topology
- 9.2 금지 Architecture
- 9.3 Topology Decision Matrix

1. Architecture 원칙과 Module Ownership

1.1 Owner 와 책임

Owner 는 기능을 실제로 소유하고 변경 책임을 지는 Module 을 뜻합니다. CPF 는 기술 Kernel, 업무 공통, 운영, Batch, Gateway, Business Domain 의 책임을 분리합니다.

- 다른 Owner 의 internal package 나 DB 를 직접 참조하지 않습니다.
- Public API/SPI 를 Consumer 가 실제 호출하는 경로로 연결합니다.

1.2 Dependency 방향

의존성은 Business Domain 에서 Public Starter/API 를 거쳐 Capability Runtime 과 cpf-core Kernel 방향으로 흐릅니다. 역방향 또는 Owner 간 순환 의존은 허용하지 않습니다.

- cpf-core 는 Spring Runtime·Admin·Batch·Gateway 구현을 소유하지 않습니다.
- Generated Domain 은 Provider/Internal leaf 를 직접 컴파일 Surface 로 노출하지 않습니다.

1.3 Public/SPI/Internal 경계

Public API 는 고객 Application 이 직접 사용할 안정 계약이고 SPI 는 확장 구현 계약입니다. Internal type 은 Public BOM·Generated Domain compile surface 에 노출하지 않습니다.

- 확장점에는 lifecycle·호환성·실패 의미를 함께 정의합니다.
- Native Spring/OSS API 가 더 적합한 경우 escape hatch 를 유지합니다.

Module 별 Owner 책임과 직접 참조 금지 경계를 확인합니다.

표 1-1. 영역/Module / 책임 / 소유/경계 / 주요 Consumer

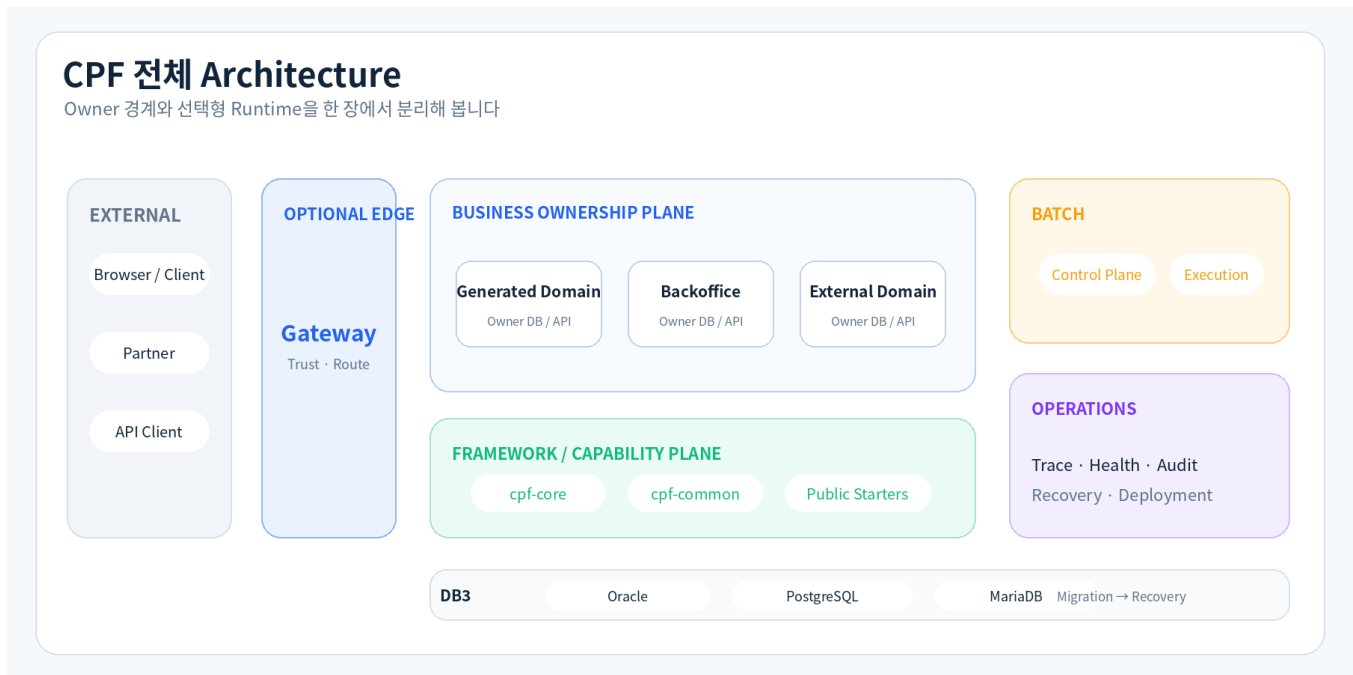
비교·선택·확인에 필요한 항목만 압축해 정리합니다.

영역/Module	책임	소유/경계	주요 Consumer
cpf-core	Kernel 의미	Runtime 구현 금지	Starter/Owner Runtime
cpf-common	고객 업무 공통	기술 Kernel 금지	Business Domain
cpf-admin	플랫폼 운영	업무 원장 금지	운영자/Frontend
cpf-batch	Batch Runtime	업무 master 금지	Scheduler/Worker
cpf-gateway	Edge/Routing	업무 로직 금지	외부 Client/Domain

[Source](#) [길찾기](#) [settings.gradle](#)

2. CPF 전체 Architecture Map

그림 2-1. CPF 전체 Architecture



Business Ownership, Framework Capability, Optional Edge, Batch, Operations, DB3 를 분리합니다.

2.1 Business Domain

Generated Business Domain 은 업무 기능과 Owner DB 를 소유하고 CPF Public API/Starter 를 통해 공통 기능을 사용합니다. 다른 Domain 의 internal package 나 DB 를 직접 참조하지 않습니다.

- 민감정보 전송·보존 정책을 Provider 경계에서 적용합니다.
- Business Domain 은 자신의 업무 원장과 규칙을 소유하고 다른 Domain 의 Internal/DB 를 직접 참조하지 않습니다.

2.2 Framework/Starter/Common

Starter 는 Application 이 필요한 Capability 를 명시적으로 선택하는 Public Entry 입니다. Provider 는 명시 선택하고 충돌 시 fail-fast 하며 Internal leaf 는 개발자 선택 Surface 로 노출하지 않습니다.

- 한 Deployable 은 기본적으로 exactly-one Top-level Profile 을 선택합니다.
- 선택하지 않은 Optional Capability 는 bean/thread/config/sql/endpoint Side Effect 가 없어야 합니다.

2.3 Gateway/Backoffice

cpf-backoffice 는 Optional Prebuilt Business Domain 이고 cpf-backoffice-web 은 DB-less Channel/BFF Reference 입니다. Browser 는 Channel 을 통해 호출하고 Business 원장은 Owner Domain 이 유지합니다.

- Backoffice 가 Member 등 다른 Domain DB 를 직접 조회하지 않습니다.
- 선택형 Backoffice 를 제거해도 core/common/admin/gateway/다른 Domain 이 정상 동작해야 합니다.

2.4 Batch

Batch 는 cpf-batch 가 Control Plane·Scheduler·Worker·Center-Cut·Agent Runtime 을 소유하고 업무 Domain 의 batch module 은 Job 정의와 업무 처리를 연결합니다. 실행은 LOCAL·PARALLEL_STEPS·LOCAL_PARTITION 및 DB 기반 claim/lease/fencing 을 사용하며 Batch 전용 Kafka Remote Execution 은 사용하지 않습니다.

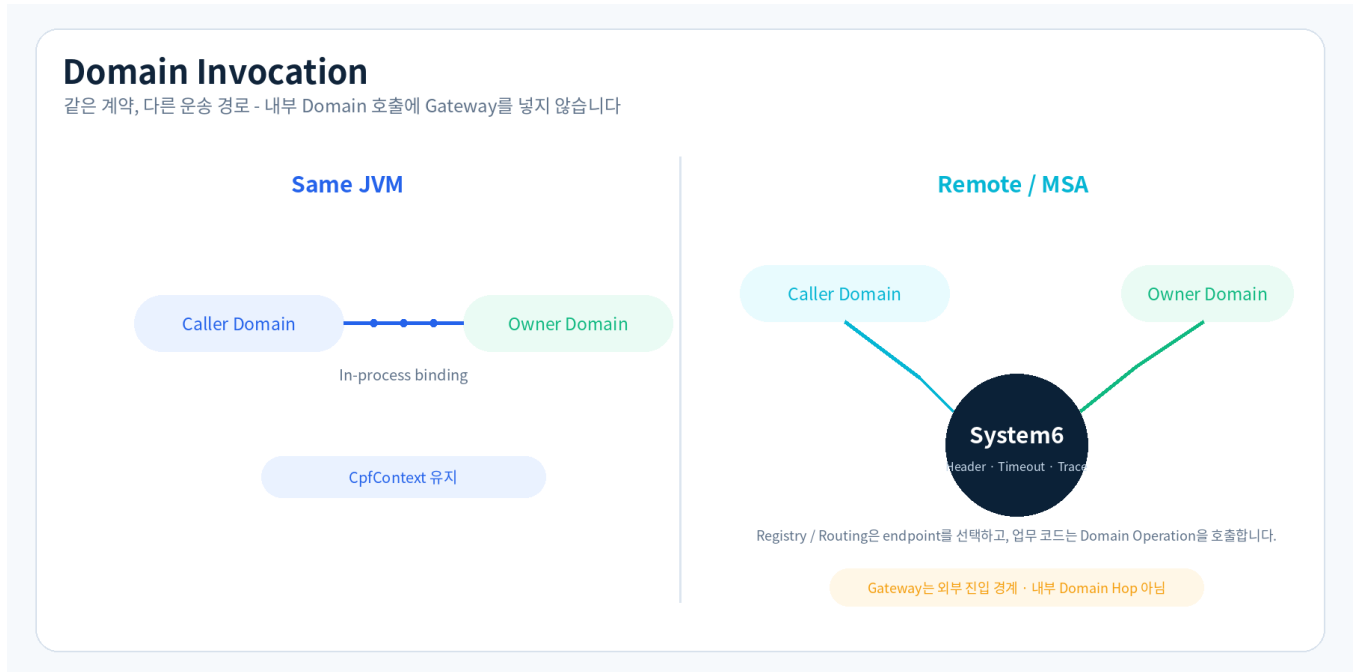
2.5 DB/Operations

DB 는 Owner 별 원장을 유지하면서 DB3 lifecycle 을 따르고 Operations 는 transaction·operation·instance 축으로 상태와 Trace 를 조회합니다. 운영 계층이 업무 원장을 직접 소유하지 않습니다.

Source [길찾기 cpf-tools/db](#)

3. Same JVM·Remote·Domain Invocation

그림 3-1. Domain Invocation



배포 방식과 무관하게 Domain 계약을 유지하고 내부 호출에 Gateway 를 강제하지 않습니다.

3.1 Same JVM

업무 Source 는 배치 방식에 따라 local/remote 분기를 작성하지 않습니다. 공식 Domain Invocation 계층이 Same JVM 과 Remote route 를 선택하면서 동일한 operation·context·error 의미를 유지합니다.

- Same JVM 은 CpfContext 계열 논리 Context 를 사용하고 self-HTTP 를 하지 않습니다.
- Remote 는 System6, timeout/deadline, retry, trace 와 UNKNOWN/reconcile 계약을 경계에서 적용합니다.

3.2 Remote/MSA

Remote 호출은 Registry 와 Routing 이 endpoint 를 선택하고 CPF 가 경계 Context 를 직렬화합니다. 호출자는 endpoint 위치가 아니라 Domain/Operation 계약을 기준으로 호출합니다.

- timeout/deadline budget 을 초과한 재시도는 금지합니다.

- 결과를 확정할 수 없으면 UNKNOWN 으로 남겨 Reconcile 경로로 연결합니다.

3.3 Domain Invocation

Domain Invocation 은 호출자가 local/remote transport 를 직접 분기하지 않게 하고 대상 해석, Context 전파, timeout/error 의미를 하나의 Public Contract 로 유지합니다.

- Local route 는 동일 JVM binding 을 사용하고 원격 transport 를 거치지 않습니다.
- Remote route 는 System6·deadline·retry·trace 와 UNKNOWN/Reconcile 계약을 transport 경계에서 적용합니다.

3.4 Self-HTTP 금지

동일 JVM 에서 자기 Domain 이나 다른 Domain 을 HTTP 로 재진입하는 self-HTTP 는 금지합니다. 같은 호출 의미는 Domain Invocation 의 local route 가 처리합니다.

- HTTP 우회로 Security/Trace/Transaction 의미를 중복 구현하지 않습니다.
- 배포 형태가 바뀌어도 업무 Service 코드는 변경하지 않는 것을 목표로 합니다.

배포 방식이 달라도 유지해야 할 호출 계약을 비교합니다.

표 3-1. 관점 / Same JVM / Remote / 공통 계약

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

관점	Same JVM	Remote	공통 계약
호출 경로	In-process route	Registry+transport	Domain Invocation
Context	CpfContext	System6 serialize	동일 논리 Context
실패	직접 예외/Result	timeout·network	표준 Error/UNKNOWN
관측	동일 trace	hop trace	transaction/operation 유지

Source 길찾기 [cpf-core/src/main/java/com/cpf/core/api/domain/CpfDomainClient.java](#)

4. Context·Security Boundary

4.1 System6

업무 Online Transaction 은 Controller 실행 전에 Canonical System6 를 갖습니다. Transaction/Original 은 유지하고 System/Caller/Target/Operation 은 Hop 기준으로 CPF 가 확정합니다.

- Browser 가 Protected Header 를 보내도 신뢰하지 않고 Trusted Entry 에서 재구성합니다.
- 수신 System/Target/Operation 이 실제 Runtime/Handler 와 다르면 업무 Controller 를 호출하지 않습니다.

4.2 Trust/Security Boundary

신뢰 경계에서는 호출자가 보낸 Protected Header 와 논리 System identity 를 분리합니다. Gateway/BFF 는 Registry·Runtime 설정으로 검증된 SystemCode 를 사용합니다.

- 미등록·비활성·spoofed identity 는 ALL 정책이어도 허용하지 않습니다.
- 관리 API 에는 업무 System6 를 억지 적용하지 않고 일반 Security/Trace/Audit 를 적용합니다.

4.3 Operation/Instance Identity

operationId 는 Annotation, OpenAPI, Domain Client, Registry, ADM, Log/Trace 가 공유하는 안정 실행 정의 ID 입니다. 개별 실행 건의 executionId 와 분리합니다.

- Source 는 operation metadata 를, ADM Policy 는 enabled/allowlist/override 를 소유합니다.
- Discovery 에서 일시 미발견된 Operation 을 자동 삭제하지 않습니다.

각 Header 의 설정 주체·변경 규칙·수신 검증을 확인합니다.

표 4-1. Header / 설정 주체 / 변경 규칙 / 검증/주의

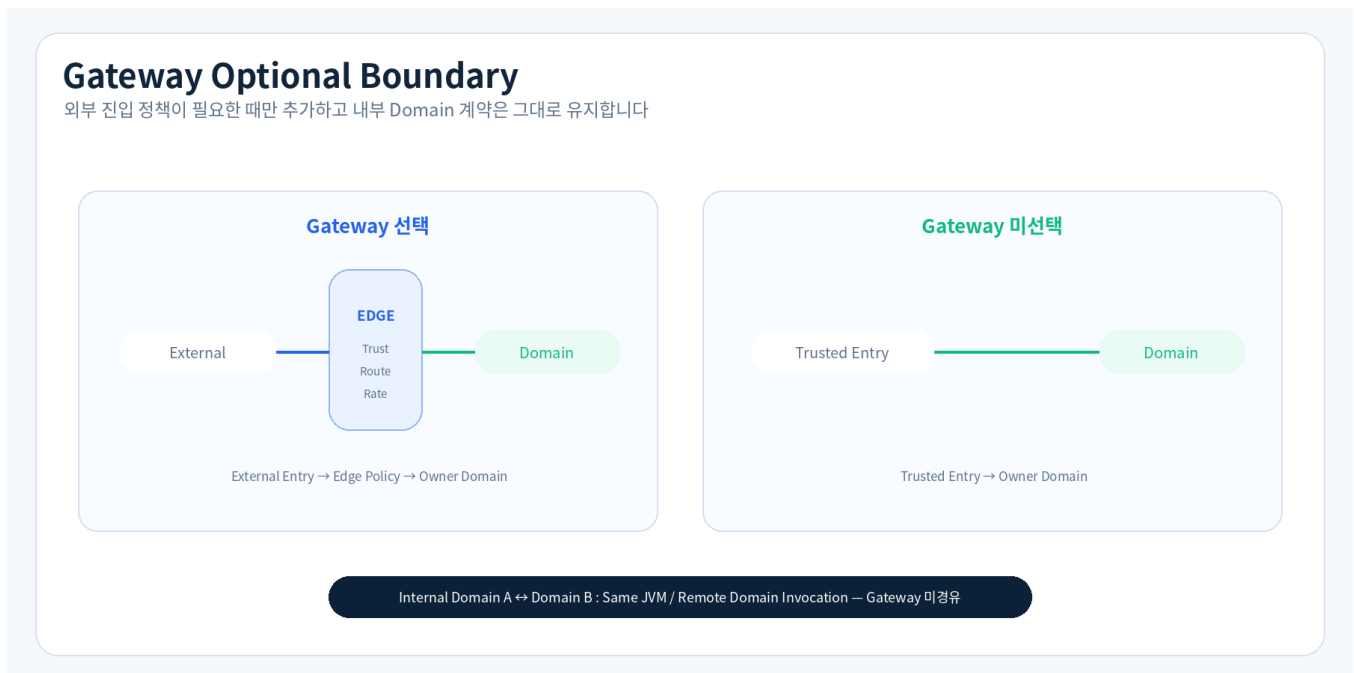
비교·선택·확인에 필요한 항목만 압축해 정리합니다.

Header	설정 주체	변경 규칙	검증/주의
X-Transaction-Id	CPF	최초 생성 후 불변	거래 전체 correlation
X-Original-System-Code	CPF Trusted Entry	최초 확정 후 불변	호출자가 임의 변경 금지
X-System-Code	CPF	Hop 마다 현재 처리 System	수신 Runtime 과 일치
X-Caller-System-Code	CPF	Hop 마다 직전 Caller	신뢰 System identity 사용
X-Target-System-Code	CPF	Hop 마다 Target	수신 Runtime 과 일치
X-Target-Operation-Id	CPF	Operation 마다 갱신	실제 Handler 와 일치

Source 길찾기 [cpf-core/src/main/java/com/cpf/core/api/context/CpfContext.java](#)

5. Gateway·Backoffice

그림 5-1. Gateway Optional Boundary



외부 진입 정책만 Gateway 에 두고 업무 Owner 경계는 Domain 에 남깁니다.

5.1 Gateway Optionality

Gateway 는 외부 진입·trust normalization·routing·rate/resilience 를 담당하는 Optional Edge 입니다. 업무 로직과 Business DB 는 Gateway 에 두지 않습니다.

- Gateway 없이 배포하는 topology 도 공식 Domain 계약을 유지합니다.
- Route 변경은 health/draining/canary/rollback 과 함께 운영합니다.

5.2 Backoffice/Owner Domain 경계

Backoffice 는 사용자/운영 요청을 Channel/BFF 경계에서 받아 공식 Domain Invocation 으로 Owner Domain 에 전달합니다. Backoffice 가 Business 원장을 직접 조회·수정하지 않습니다.

상황에 맞는 선택과 피해야 할 경로를 판단합니다.

Module 별 Owner 책임과 직접 참조 금지 경계를 확인합니다.

표 5-1. 영역/Module / 책임 / 소유/경계 / 주요 Consumer

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

영역/Module	책임	소유/경계	주요 Consumer
cpf-core	Kernel 의미	Runtime 구현 금지	Starter/Owner Runtime
cpf-common	고객 업무 공통	기술 Kernel 금지	Business Domain
cpf-admin	플랫폼 운영	업무 원장 금지	운영자/Frontend
cpf-batch	Batch Runtime	업무 master 금지	Scheduler/Worker
cpf-gateway	Edge/Routing	업무 로직 금지	외부 Client/Domain

[Source 길찾기 cpf-backoffice](#)

6. Batch Architecture

그림 6-1. Batch Architecture



제어와 실행을 분리해 Lease/Fencing 과 Process Kill 복구를 운영 계약으로 만듭니다.

6.1 Control Plane/Scheduler/Worker

Batch Runtime 은 Control Plane, Scheduler, Worker, Center-Cut, Agent 의 역할을 분리합니다. 제어 명령과 실제 실행 상태를 같은 Process 내부 변수로만 관리하지 않으며 Batch 전용 Kafka/Broker Remote Execution 을 사용하지 않습니다.

- Scheduler 는 실행 계획/trigger 를, Worker 는 실제 Job 실행을 소유합니다.
- Center-Cut/Agent 는 대상 분할·Process 실행과 lease/fencing 상태를 추적합니다.

6.2 Center-Cut/Agent

Center-Cut 은 대량 대상 분할·집계를 관리하고 Agent 는 Job Pack/Process 실행 경계를 담당하며 Batch 전용 Broker Remote Transport 는 사용하지 않습니다. lease/fencing/heartbeat 로 다중 인스턴스 소유권을 보호합니다.

- Scheduler 는 실행 계획/trigger 를, Worker 는 실제 Job 실행을 소유합니다.
- Center-Cut/Agent 는 대상 분할·Process 실행과 lease/fencing 상태를 추적합니다.

6.3 Lease/Fencing/Heartbeat

Lease 는 작업 소유권의 유효기간이고 Fencing 은 stale Worker 의 늦은 쓰기를 차단하는 토큰입니다. Heartbeat 만 살아 있다고 소유권을 무기한 인정하지 않습니다.

- lease 만료 후 재할당 시 fencing token 이 증가해야 합니다.
- clock/timeout 경계와 DB update 조건을 동시성 Test 로 검증합니다.

각 역할의 책임과 장애 시 영향을 구분합니다.

표 6-1. 역할 / 책임 / 소유/경계 / 장애 영향

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

역할	책임	소유/경계	장애 영향
Control Plane	정의·제어·상태	제어 Owner	명령/상태 불일치
Scheduler	Trigger·할당	실행 계획	중복 scheduling
Worker	Job 실행	execution	Process kill
Center-Cut	대상 분할·조정	대량 실행	부분 실패
Agent	Job Pack/Process 실행 ·heartbeat	instance/lease	stale agent

Source 길찾기 [cpf-batch/api/src/main/java/com/cpf/batch/spi/BatchFencingPort.java](#)

7. Persistence·DB Ownership

7.1 DB Owner

Business 원장은 해당 Owner Domain 이 소유하고 Admin·Gateway·Backoffice 가 직접 DB 접근으로 우회하지 않습니다. DB 변경은 Owner 와 migration lifecycle 을 함께 따라야 합니다.

7.2 Oracle/PostgreSQL/MariaDB

CPF 의 공식 DB Vendor 는 Oracle, PostgreSQL, MariaDB 입니다. 하나의 Canonical DB 의미를 기준으로 Vendor 별 DDL·runtime query 차이를 명시적으로 render/검증합니다.

공식 DB Vendor 는 Oracle·PostgreSQL·MariaDB 만 사용하며 세 Vendor 의 DDL·Migration·Rollback·Runtime 동작을 함께 검증합니다.

- Vendor 차이는 datatype·identity·locking·DDL syntax 처럼 실제 동작 차이가 있는 지점에서만 분기합니다.

7.3 Migration/Upgrade/Rollback 경계

DB 변경은 Canonical Source→Fresh Init→Vendor

Render→Migration→Seed→Upgrade→Rollback/Recovery→Runtime Query→Test 를 하나의 Lifecycle 로 단습니다.

- DDL 한 파일 변경만으로 완료 처리하지 않습니다.
- Rollback 이 위험하거나 불가능하면 명시적 recovery 절차와 Evidence 를 남깁니다.

Oracle·PostgreSQL·MariaDB 의 공통 의미와 검증 지점을 비교합니다.

표 7-1. 항목 / Oracle / PostgreSQL / MariaDB

비교·선택·확인에 필요한 항목만 압축해 정리합니다.

항목	Oracle	PostgreSQL	MariaDB
공식 지원	지원	지원	지원
Canonical 의미	공통	공통	공통

항목	Oracle	PostgreSQL	MariaDB
Vendor Render	oracle	postgresql	mariadb
검증	DDL·Runtime	DDL·Runtime	DDL·Runtime

DB 변경이 Fresh Init 부터 Runtime Query 까지 단렸는지 확인합니다.

표 7-2. 단계 / Canonical / Vendor 영향 / 검증/복구

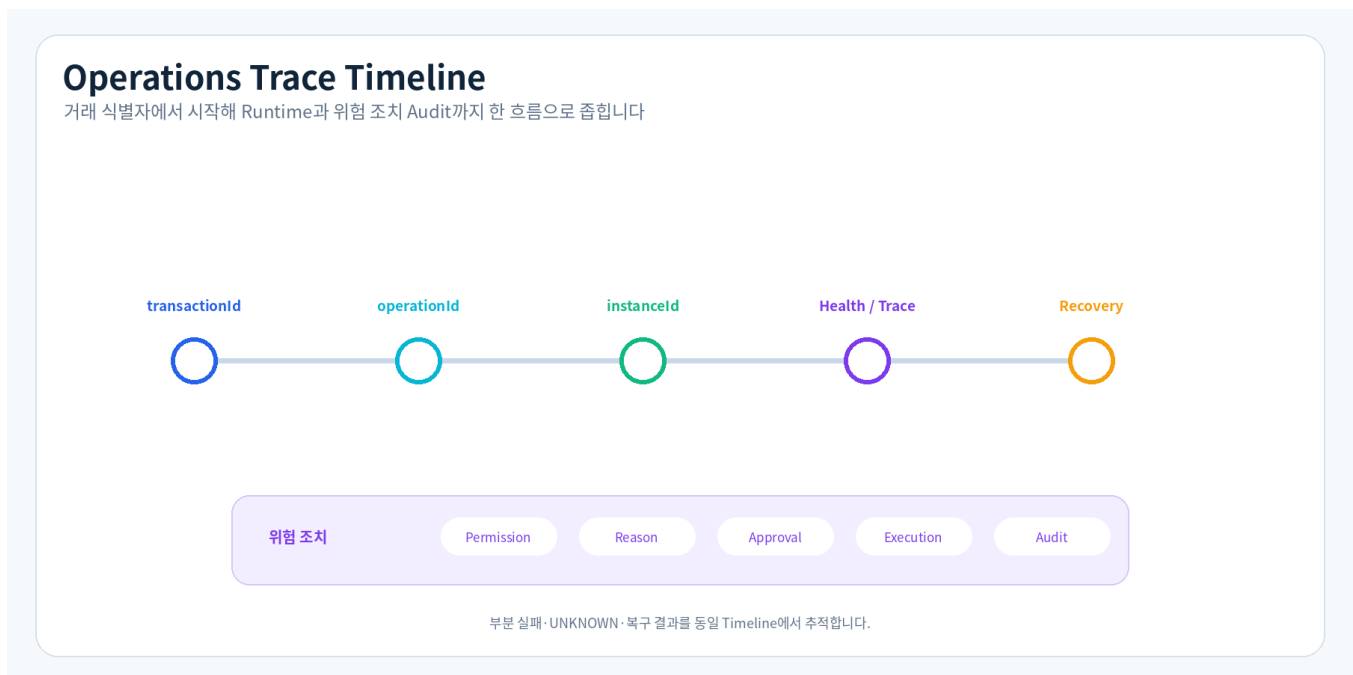
비교·선택·확인에 필요한 항목만 압축해 정리합니다.

단계	Canonical	Vendor 영향	검증/복구
Fresh Init	Canonical schema	Vendor 별 DDL	깨끗한 초기화
Migration	변경 정의	문법/락 차이	upgrade 실행
Seed	공통 기준 데이터	DML 차이	재실행 안전성
Rollback/Recovery	역변경/복구	Vendor 별 절차	실패 복구 Evidence
Runtime Query	Repository 의미	SQL 차이	실거래 검증

[Source](#) [길찾기](#) [cpf-tools/db](#)

8. Observability·Multi-instance·HA

그림 8-1. Runtime Trace



거래·Operation·Instance 식별자로 다중 인스턴스 장애를 추적합니다.

8.1 Log/Trace/Timeline

Observability 는 transactionId · operationId · instanceId 를 중심으로 Log, Trace, Metric, Timeline 을 연결합니다. 운영자는 코드 위치보다 거래 · 실행 식별자에서 시작해 원인을 좁힐 수 있어야 합니다.

- 민감정보는 structured log 에서도 마스킹합니다.
- 재시도/복구 segment 가 원 거래와 단절되지 않도록 correlation 을 유지합니다.

8.2 Multi-instance

instanceId 는 현재 Process 를 식별하는 Runtime 축이며 System6 Header 에는 포함하지 않습니다. 기동 시 설정 → 환경 변수 → Hostname 순으로 한 번 확정하고 Process 수명 동안 유지합니다.

- localhost/unknown/Domain 명 같은 fallback 으로 READY 를 허용하지 않습니다.
- 동일 Host 의 동일 System 다중 Process 는 explicit instanceId 가 필요합니다.

8.3 HA/Failure Domain

HA 는 System 전체를 하나의 생존 여부로 보지 않고 instance · provider · DB · Gateway · Worker 등 Failure Domain 별로 격리합니다. 한 instance 장애가 다른 Owner 의 상태를 임의 변경하지 않습니다.

- 민감정보 전송 · 보존 정책을 Provider 경계에서 적용합니다.
- Failover 는 해당 Failure Domain 안에서 수행하며 보안 · Owner 경계를 자동 우회하지 않습니다.

Source [길찾기](#) `cpf-starters/base/runtime/src/main/java/com/cpf/foundation/runtime/CpfInstanceIdentity.java`

9. Deployment · 금지 패턴 · Decision

9.1 Deployment Topology

CPF 는 Same JVM, 분리 WAS, MSA, 다중 인스턴스 topology 에서 동일 Domain 계약을 유지합니다. 배포 형태가 바뀌어도 업무 Source 가 local/remote 분기를 직접 구현하지 않습니다.

9.2 금지 Architecture

금지 패턴은 Owner · Topology · 신뢰 경계를 우회하거나 운영 Evidence 를 잃게 만드는 구현을 빠르게 차단하기 위한 Review Gate 입니다.

- self-HTTP, 타 Owner DB/Internal 직접 참조, secret 원문 로그를 금지합니다.
- 미실행 검증 · stale Evidence · Optional 기능의 숨은 Side Effect 를 완료 근거로 사용하지 않습니다.

9.3 Topology Decision Matrix

Topology 는 latency, 장애 격리, 독립 배포, 운영 복잡도와 데이터 Owner 경계를 기준으로 선택합니다.

Gateway/Backoffice 같은 Optional Runtime 은 제거해도 핵심 Domain 계약이 유지되어야 합니다.

상황에 맞는 선택과 피해야 할 경로를 판단합니다.

Source [길찾기](#) `settings.gradle`